Assignment 4 — California Housing

SVNIT — Department of Artificial Intelligence

Question sheet: `ML-04.pdf`
Dataset: `../data/housing.csv` (StatLib 1990 census)

Plots are written to `../plots/assignment_4/` (not shown inline).

In [1]:

```
import warnings
from pathlib import Path
from urllib.request import urlretrieve

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score, root_mean_s
quared_error
from sklearn.model_selection import GridSearchCV, StratifiedShuffleSpl
it, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.tree import DecisionTreeRegressor

warnings.filterwarnings("ignore")
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")
pd.set_option("display.max_columns", 20)
pd.set_option("display.width", 120)

HOUSING_URL = (
    "https://raw.githubusercontent.com/ageron/handson-ml2/master/datas
ets/housing/housing.csv"
)


def find_project_root():
    starts = [Path.cwd().resolve()]
    for key in ("__vsc_ipynb_file__", "__session__"):
        val = globals().get(key)
        if val:
            starts.append(Path(val).resolve().parent)
    seen = set()
    for start in starts:
        for p in [start, *start.parents]:
            if p in seen:
                continue
            seen.add(p)
            if (p / "data").is_dir() and (
                (p / "data" / "housing.csv").exists() or (p / "data" /
"train.csv").exists()
            ):
                return p
    raise FileNotFoundError("Could not find the ML Lab data/ folder.")


ROOT = find_project_root()
DATA = ROOT / "data"
PLOTS = ROOT / "plots" / "assignment_4"
PLOTS.mkdir(parents=True, exist_ok=True)
```

```
housing_path = DATA / "housing.csv"
if not housing_path.exists():
    print("Downloading California Housing dataset...")
    urlretrieve(HOUSING_URL, housing_path)


def section(title):
    print("\n" + "=" * 64)
    print(f"  {title}")
    print("=" * 64)


def subsection(title):
    print(f"\n--- {title} ---")


def pct(value, digits=2):
    return f"{100 * value:.{digits}f}%"


def money(value):
    return f"${value:,.2f}"


def save_plot(name):
    path = PLOTS / name
    plt.tight_layout()
    plt.savefig(path, dpi=120, bbox_inches="tight")
    plt.close()
    print(f"Saved {path.relative_to(ROOT)}")


print("Project root:", ROOT)
print("Dataset:", housing_path)
```

```
Project root: /Users/ojas/Desktop/ML Lab
Dataset: /Users/ojas/Desktop/ML Lab/data/housing.csv
```

Q1: Stratified split on `income_cat`

In [2]:
```
section("Q1: income_cat + StratifiedShuffleSplit")

housing = pd.read_csv(DATA / "housing.csv")
print(f"Rows: {len(housing):,}   Columns: {housing.shape[1]}")
print("Columns:", list(housing.columns))

housing["income_cat"] = pd.cut(
    housing["median_income"],
    bins=[0.0, 1.5, 3.0, 4.5, 6.0, np.inf],
    labels=[1, 2, 3, 4, 5],
)

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state
=42)
for train_idx, test_idx in split.split(housing, housing["income_ca
t"]):
    train_set = housing.loc[train_idx].copy()
    test_set = housing.loc[test_idx].copy()

print(f"\nTrain size: {len(train_set):,}   Test size: {len(test_se
t):,}")

train_counts = train_set["income_cat"].value_counts().sort_index()
test_counts = test_set["income_cat"].value_counts().sort_index()
overall = housing["income_cat"].value_counts().sort_index()

counts = pd.DataFrame({
    "train_count": train_counts,
    "test_count": test_counts,
    "train_share": train_counts / len(train_set),
    "test_share": test_counts / len(test_set),
    "overall_share": overall / len(housing),
})
counts.index.name = "income_cat"

print("\nCount of each income_cat in train and test:")
print(f"{'income_cat':<12} {'train':>8} {'test':>8} {'train %':>10}
{'test %':>10} {'overall %':>12}")
print("-" * 64)
for cat, row in counts.iterrows():
    print(
        f"{int(cat):<12} {int(row['train_count']):>8} {int(row['test_c
ount']):>8} "
        f"{pct(row['train_share']):>10} {pct(row['test_share']):>10}
{pct(row['overall_share']):>12}"
    )

print("\nShares match the overall distribution (stratify worked).")
```

```
============================================================
  Q1: income_cat + StratifiedShuffleSplit
============================================================
Rows: 20,640   Columns: 10
Columns: ['longitude', 'latitude', 'housing_median_age', 'total_room
s', 'total_bedrooms', 'population', 'households', 'median_income', 'me
dian_house_value', 'ocean_proximity']

Train size: 16,512   Test size: 4,128

Count of each income_cat in train and test:
income_cat      train     test    train %     test %    overall %
------------------------------------------------------------------
1                 657      165      3.98%      4.00%        3.98%
2                5265     1316     31.89%     31.88%       31.88%
3                5789     1447     35.06%     35.05%       35.06%
4                2911      728     17.63%     17.64%       17.63%
5                1890      472     11.45%     11.43%       11.44%

Shares match the overall distribution (stratify worked).
```

Q2: `ColumnTransformer` (median impute + scale, one-hot `ocean_proximity`)

In [3]:

```
section("Q2: Preprocessing pipeline")

# income_cat is only for splitting — drop it before modeling.
train_set = train_set.drop(columns=["income_cat"])
test_set = test_set.drop(columns=["income_cat"])

X_train = train_set.drop(columns=["median_house_value"])
y_train = train_set["median_house_value"].copy()
X_test = test_set.drop(columns=["median_house_value"])
y_test = test_set["median_house_value"].copy()

num_attribs = X_train.select_dtypes(include=[np.number]).columns.tolis
t()
cat_attribs = ["ocean_proximity"]
print("Numeric features:", num_attribs)
print("Categorical features:", cat_attribs)
print(f"Missing total_bedrooms in train: {X_train['total_bedrooms'].is
na().sum()}")

num_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

preprocess = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_attribs),
])

X_train_prep = preprocess.fit_transform(X_train)
if hasattr(X_train_prep, "toarray"):
    X_train_prep = X_train_prep.toarray()

feature_names = preprocess.get_feature_names_out()
print(f"\nPrepared training array shape: {X_train_prep.shape}")
print(f"(rows = train houses, columns = {len(num_attribs)} scaled nume
ric + "
      f"{X_train_prep.shape[1] - len(num_attribs)} one-hot ocean_proxi
mity)")
print("Feature names:")
for name in feature_names:
    print(f"  {name}")
```

```
============================================================
  Q2: Preprocessing pipeline
============================================================
Numeric features: ['longitude', 'latitude', 'housing_median_age', 'tot
al_rooms', 'total_bedrooms', 'population', 'households', 'median_incom
e']
Categorical features: ['ocean_proximity']
Missing total_bedrooms in train: 158

Prepared training array shape: (16512, 13)
(rows = train houses, columns = 8 scaled numeric + 5 one-hot ocean_pro
ximity)
Feature names:
  num__longitude
  num__latitude
  num__housing_median_age
  num__total_rooms
  num__total_bedrooms
  num__population
  num__households
  num__median_income
  cat__ocean_proximity_<1H OCEAN
  cat__ocean_proximity_INLAND
  cat__ocean_proximity_ISLAND
  cat__ocean_proximity_NEAR BAY
  cat__ocean_proximity_NEAR OCEAN
```

Q3: LinearRegression, DecisionTree, RandomForest — 5-fold CV RMSE

In [4]:
```
section("Q3: Cross-validated RMSE")

models = {
    "LinearRegression()": LinearRegression(),
    "DecisionTreeRegressor(random_state=42)": DecisionTreeRegressor(ra
ndom_state=42),
    "RandomForestRegressor(n_estimators=100, random_state=42)": Random
ForestRegressor(
        n_estimators=100, random_state=42
    ),
}

print(f"{'Model':<56} {'CV RMSE (mean ± std)':>28}")
print("-" * 86)
for name, model in models.items():
    scores = -cross_val_score(
        model,
        X_train_prep,
        y_train,
        scoring="neg_root_mean_squared_error",
        cv=5,
    )
    mean_rmse, std_rmse = scores.mean(), scores.std()
    print(f"{name:<56} {money(mean_rmse)} ± {money(std_rmse)}")

print("\nLower RMSE is better. Random Forest should beat the single tr
ee "
      "(less overfitting) and linear regression (non-linear splits).")
```

```
============================================================
  Q3: Cross-validated RMSE
============================================================
Model                                                   CV RM
SE (mean ± std)
------------------------------------------------------------------------
----------------
LinearRegression()                                    $69,218.45 ±
$689.50

DecisionTreeRegressor(random_state=42)                $70,629.24 ±
$1,447.66

RandomForestRegressor(n_estimators=100, random_state=42) $49,941.73 ±
$760.90

Lower RMSE is better. Random Forest should beat the single tree (less
overfitting) and linear regression (non-linear splits).
```

Q4: `GridSearchCV` on RandomForest

In [5]:
```
section("Q4: GridSearchCV on RandomForestRegressor")

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_features": [2, 4, 6, 8],
}

grid = GridSearchCV(
    RandomForestRegressor(random_state=42),
    param_grid,
    cv=5,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1,
    return_train_score=False,
)
grid.fit(X_train_prep, y_train)

best_cv_rmse = -grid.best_score_
print(f"Best hyperparameters: {grid.best_params_}")
print(f"Best CV RMSE:         {money(best_cv_rmse)}")

print("\nAll grid combinations (ranked by CV RMSE):")
results = pd.DataFrame(grid.cv_results_)
results["cv_rmse"] = -results["mean_test_score"]
show = results[["param_n_estimators", "param_max_features", "cv_rms
e"]].sort_values("cv_rmse")
print(f"{'n_estimators':>14} {'max_features':>14} {'CV RMSE':>14}")
print("-" * 44)
for _, row in show.iterrows():
    print(f"{int(row['param_n_estimators']):>14} {int(row['param_max_f
eatures']):>14} {money(row['cv_rmse']):>14}")
```

```
============================================================
  Q4: GridSearchCV on RandomForestRegressor
============================================================

Best hyperparameters: {'max_features': 8, 'n_estimators': 200}
Best CV RMSE:         $49,015.19

All grid combinations (ranked by CV RMSE):
  n_estimators   max_features        CV RMSE
-------------------------------------------
           200              8     $49,015.19
           100              8     $49,153.15
           200              6     $49,160.20
           100              6     $49,358.35
           200              4     $49,415.26
            50              8     $49,477.45
           100              4     $49,581.33
            50              6     $49,650.24
            50              4     $49,990.89
           200              2     $51,227.60
           100              2     $51,515.44
            50              2     $52,062.68
```

Q5: Held-out test RMSE, MAE, R²

In [6]:
```
section("Q5: Test-set evaluation")

X_test_prep = preprocess.transform(X_test)
if hasattr(X_test_prep, "toarray"):
    X_test_prep = X_test_prep.toarray()

best_rf = grid.best_estimator_
y_pred = best_rf.predict(X_test_prep)

test_rmse = root_mean_squared_error(y_test, y_pred)
test_mae = mean_absolute_error(y_test, y_pred)
test_r2 = r2_score(y_test, y_pred)
mean_y = y_test.mean()

print(f"{'Metric':<28} {'Absolute':>16} {'As %':>18}")
print("-" * 64)
print(f"{'Test RMSE':<28} {money(test_rmse):>16} {(test_rmse / mean_y)
* 100:>16.2f}% of mean")
print(f"{'Test MAE':<28} {money(test_mae):>16} {(test_mae / mean_y) *
100:>16.2f}% of mean")
print(f"{'Test R²':<28} {test_r2:>16.4f} {pct(test_r2):>18}")
print(f"{'Mean test house value':<28} {money(mean_y):>16}")
print(f"\nR² = {pct(test_r2)} of test-set variance explained. "
      f"Typical miss (RMSE) is {money(test_rmse)}.")
```

```
================================================================
  Q5: Test-set evaluation
================================================================
Metric                                Absolute                As %
----------------------------------------------------------------
Test RMSE                           $46,563.78            22.58% of mea
n
Test MAE                            $30,805.89            14.94% of mea
n
Test R²                                 0.8336             83.36%
Mean test house value              $206,257.80

R² = 83.36% of test-set variance explained. Typical miss (RMSE) is $4
6,563.78.
```

Q6: Normal Equation vs `LinearRegression` (NumPy only)

In [7]:
```
section("Q6: Normal Equation (NumPy)")

# Two raw features + bias column. No scaling — both methods see the sa
me matrix.
X_two = train_set[["median_income", "housing_median_age"]].to_numpy(dt
ype=float)
y_two = y_train.to_numpy(dtype=float)
X_b = np.c_[np.ones(len(X_two)), X_two]  # bias / intercept column

# θ = (XᵀX)⁻¹ Xᵀ y
theta_ne = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y_two

lin = LinearRegression().fit(X_two, y_two)
theta_sk = np.r_[lin.intercept_, lin.coef_]

labels = ["θ0 (intercept / bias)", "θ1 (median_income)", "θ2 (housing_
median_age)"]

print("Hypothesis:  median_house_value = θ0 + θ1·median_income + θ2·ho
using_median_age")
print(f"\n{'Parameter':<28} {'Normal Eq':>16} {'sklearn':>16} {'% dif
f':>12}")
print("-" * 76)
for lab, a, b in zip(labels, theta_ne, theta_sk):
    diff_pct = (a - b) / b * 100
    print(f"{lab:<28} {a:>16,.4f} {b:>16,.4f} {diff_pct:>11.4e}%")

print("\nFitted (Normal Equation):")
print(
    f"  median_house_value = {theta_ne[0]:,.2f} "
    f"+ {theta_ne[1]:,.2f}×median_income "
    f"+ {theta_ne[2]:,.2f}×housing_median_age"
)
print("\n% difference is ~0 (floating-point noise). Both methods solve
the same OLS problem.")
print("Interpretation: +1 in median_income (~+$10k household income) →
"
      f"{money(theta_ne[1])} median house value, holding age fixed.")
```

```
============================================================
  Q6: Normal Equation (NumPy)
============================================================
Hypothesis:  median_house_value = θ0 + θ1·median_income + θ2·housing_m
edian_age

Parameter                          Normal Eq          sklearn       %
diff
------------------------------------------------------------------------
------
θ0 (intercept / bias)            -10,706.8909     -10,706.8909 -1.1060
e-11%
θ1 (median_income)                43,041.0775      43,041.0775 -2.5357
e-13%
θ2 (housing_median_age)            1,776.0535       1,776.0535 -1.1266
e-12%

Fitted (Normal Equation):
  median_house_value = -10,706.89 + 43,041.08×median_income + 1,776.05
×housing_median_age

% difference is ~0 (floating-point noise). Both methods solve the same
OLS problem.
Interpretation: +1 in median_income (~+$10k household income) → $43,04
1.08 median house value, holding age fixed.
```

Q7: Geographic scatter (longitude vs latitude)

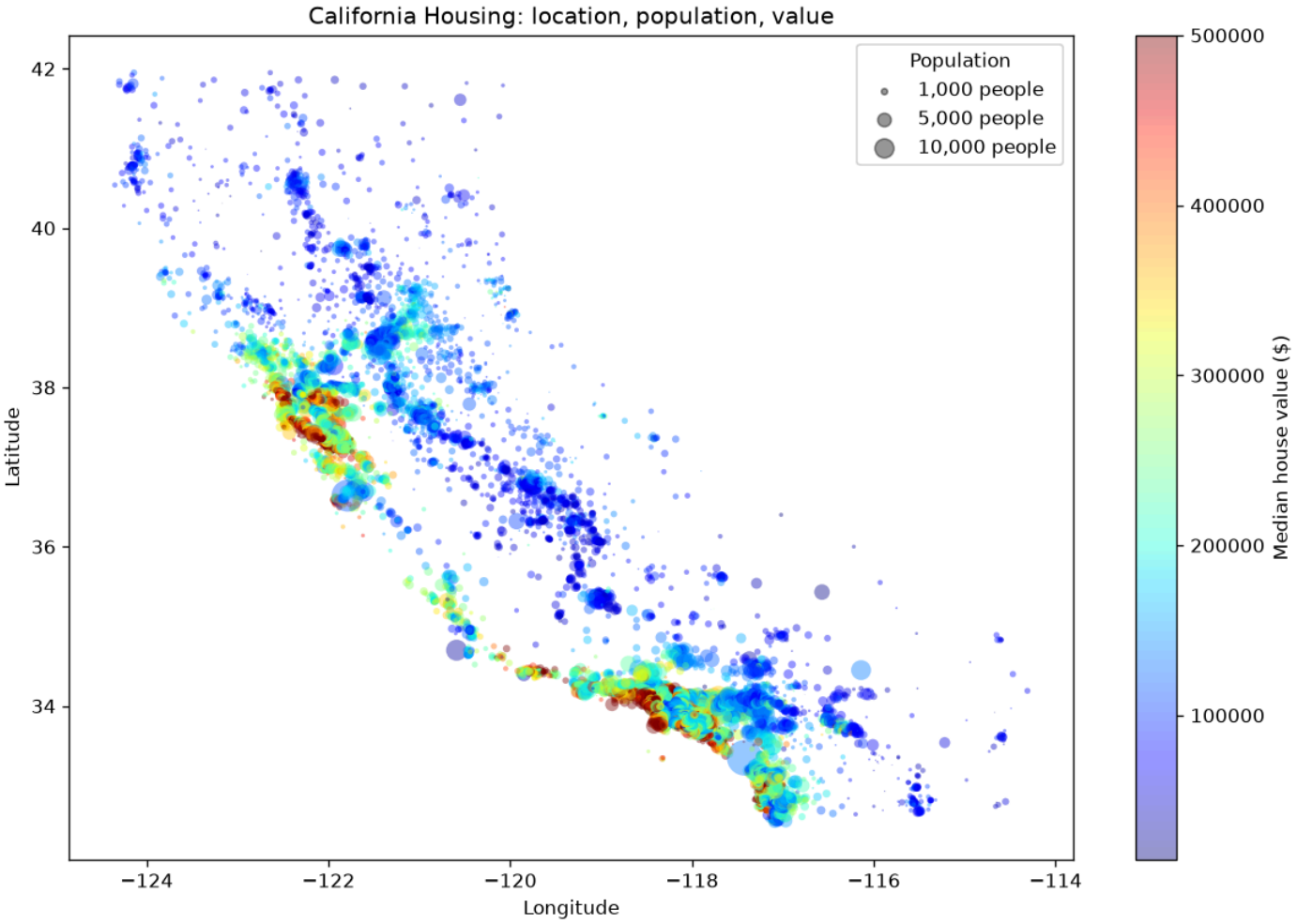
In [8]:
```
section("Q7: Geographic scatter plot")

fig, ax = plt.subplots(figsize=(10, 7))
sc = ax.scatter(
    housing["longitude"],
    housing["latitude"],
    s=housing["population"] / 100,
    c=housing["median_house_value"],
    cmap="jet",
    alpha=0.4,
    linewidths=0,
)
cbar = fig.colorbar(sc, ax=ax)
cbar.set_label("Median house value ($)")
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.set_title("California Housing: location, population, value")
for pop, size in [(1000, 10), (5000, 50), (10000, 100)]:
    ax.scatter([], [], s=size, c="k", alpha=0.4, label=f"{pop:,} peopl
e")
ax.legend(title="Population", loc="upper right", frameon=True)
save_plot("geographic_scatter.png")
print("Coastal / bay areas (left and bottom) are darker = more expensi
ve.")
```

```
============================================================
  Q7: Geographic scatter plot
============================================================

Saved plots/assignment_4/geographic_scatter.png
Coastal / bay areas (left and bottom) are darker = more expensive.
```

Geographic scatter (saved to `../plots/assignment_4/geographic_scatter.png`):



Q8: Histogram of `median_house_value`

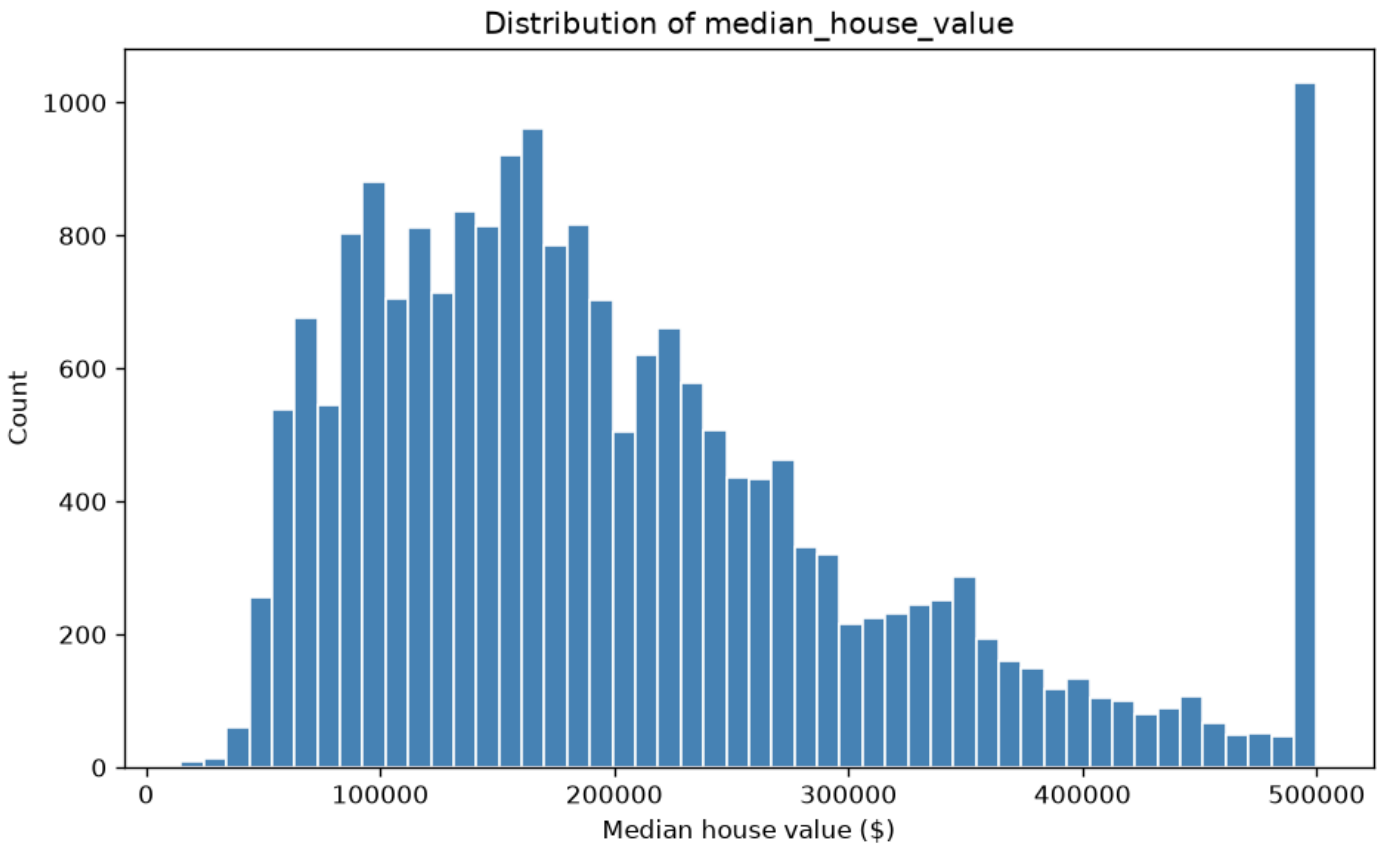
In [9]:
```
section("Q8: Histogram of median_house_value")

skew = housing["median_house_value"].skew()
print(f"Mean:     {money(housing['median_house_value'].mean())}")
print(f"Median:   {money(housing['median_house_value'].median())}")
print(f"Max:      {money(housing['median_house_value'].max())}")
print(f"Skewness: {skew:.3f}  →  {'right-skewed' if skew > 0.5 else 'a
pproximately symmetric'}")
print("Spike at $500,001 is a cap in the original census extract.")

plt.figure(figsize=(8, 5))
plt.hist(housing["median_house_value"], bins=50, color="steelblue", ed
gecolor="white")
plt.xlabel("Median house value ($)")
plt.ylabel("Count")
plt.title("Distribution of median_house_value")
save_plot("median_house_value_hist.png")
```

```
================================================================
  Q8: Histogram of median_house_value
================================================================
Mean:     $206,855.82
Median:   $179,700.00
Max:      $500,001.00
Skewness: 0.978  →  right-skewed
Spike at $500,001 is a cap in the original census extract.
Saved plots/assignment_4/median_house_value_hist.png
```

Target histogram (saved to `../plots/assignment_4/median_house_value_hist.png`):



Q9: Predicted vs actual on the test set

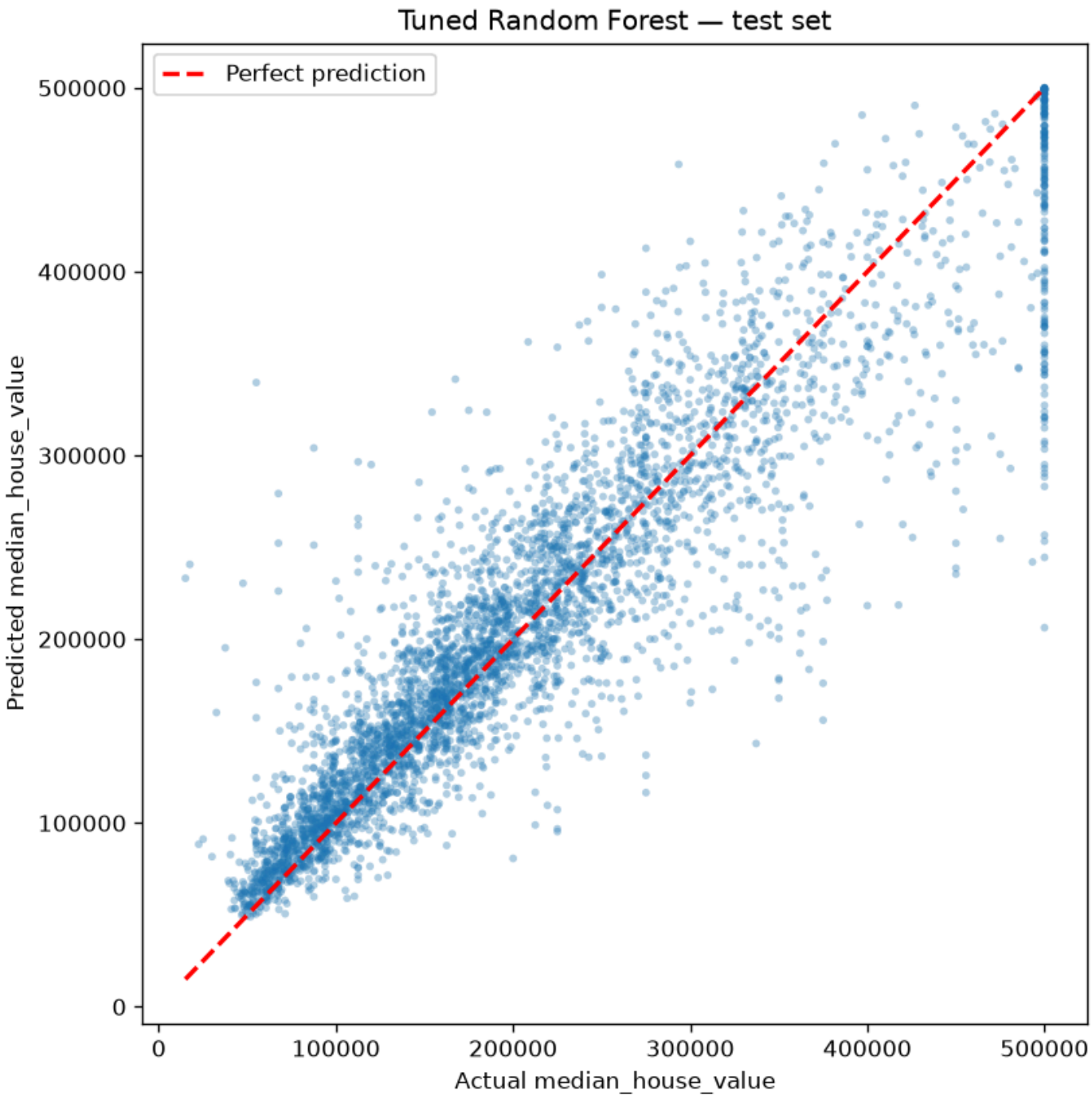
In [10]:
```
section("Q9: Predicted vs actual (test set)")

lo = min(y_test.min(), y_pred.min())
hi = max(y_test.max(), y_pred.max())

plt.figure(figsize=(7, 7))
plt.scatter(y_test, y_pred, alpha=0.35, s=12, edgecolors="none")
plt.plot([lo, hi], [lo, hi], "r--", lw=2, label="Perfect prediction")
plt.xlabel("Actual median_house_value")
plt.ylabel("Predicted median_house_value")
plt.title("Tuned Random Forest — test set")
plt.legend()
save_plot("pred_vs_actual.png")
print("Points on the red dashed line are exact. The cap at $500,001 sh
ows as a vertical strip.")
```

```
============================================================
  Q9: Predicted vs actual (test set)
============================================================
Saved plots/assignment_4/pred_vs_actual.png
Points on the red dashed line are exact. The cap at $500,001 shows as
a vertical strip.
```

Predicted vs actual (saved to `../plots/assignment_4/pred_vs_actual.png`):



Q10: Top-10 Random Forest feature importances

In [11]:
```
section("Q10: Feature importances (tuned Random Forest)")

# Strip ColumnTransformer prefixes (num__, cat__) for a readable char
t.
clean_names = [n.split("__", 1)[1] if "__" in n else n for n in featur
e_names]
imp = pd.Series(best_rf.feature_importances_, index=clean_names).sort_
values(ascending=False)
top10 = imp.head(10)

print(f"{'Rank':<6} {'Feature':<28} {'Importance':>12} {'Share':>10}")
print("-" * 58)
for i, (feat, val) in enumerate(top10.items(), start=1):
    print(f"{i:<6} {feat:<28} {val:>12.4f} {pct(val):>10}")
print(f"\nTop-10 share of total importance: {pct(top10.sum())}")

plt.figure(figsize=(8, 5))
plt.barh(top10.index[::-1], top10.values[::-1], color="steelblue")
plt.xlabel("Importance")
plt.title("Top-10 feature importances — tuned Random Forest")
save_plot("feature_importances.png")
```

```
============================================================
  Q10: Feature importances (tuned Random Forest)
============================================================
Rank   Feature                        Importance      Share
----------------------------------------------------------
1      median_income                      0.4440     44.40%
2      ocean_proximity_INLAND             0.1508     15.08%
3      longitude                          0.1144     11.44%
4      latitude                           0.1037     10.37%
5      housing_median_age                 0.0490      4.90%
6      population                         0.0374      3.74%
7      total_rooms                        0.0293      2.93%
8      total_bedrooms                     0.0267      2.67%
9      households                         0.0253      2.53%
10     ocean_proximity_<1H OCEAN          0.0109      1.09%

Top-10 share of total importance: 99.14%

Saved plots/assignment_4/feature_importances.png
```

Feature importances (saved to `../plots/assignment_4/feature_importances.png`):

